

Задание 3.6. Организация оркестрации ETL/ELT процессов хранилища с помощью Apache Airflow

В рамках данной лабораторной работы необходимо организовать централизованное управление ETL/ELT-процессами хранилища данных с помощью Apache Airflow. Оркестрация должна охватывать получение исходных данных, загрузку в слой ODS, преобразование данных для детального слоя и формирование аналитических витрин.

Разработанные DAG должны отражать реальные зависимости между этапами обработки, обеспечивать контролируемый запуск по расписанию, повторное выполнение после ошибок, инкрементальную обработку и наблюдаемость состояния конвейера.

Этапы выполнения:

1. Развернуть Apache Airflow в Docker-контейнерах и добавить сервис в общую docker-compose конфигурацию проекта. Настроить базу метаданных Airflow, веб-интерфейс, планировщик и компонент выполнения задач.
2. Создать подключения Airflow к используемым источникам и хранилищам данных. Учетные данные и параметры доступа не должны быть жестко записаны в коде DAG; необходимо использовать Connections, Variables, переменные окружения или внешний механизм хранения секретов, например HashiCorp Vault.
3. Разделить процессы хранилища на логические DAG: загрузка исходных данных, формирование детального слоя, проверки качества и построение витрин. Для однотипных операций рекомендуется использовать TaskGroup, переиспользуемые функции или собственные операторы.

Имена DAG и задач должны отражать их назначение. Код следует организовать таким образом, чтобы бизнес-логика обработки была отделена от описания порядка выполнения задач.

4. Настроить зависимости между задачами и DAG. Процессы последующего слоя должны запускаться только после успешного завершения необходимых расчетов предыдущего слоя. Допускается использование сенсоров, наборов данных Airflow, триггеров или единого управляющего DAG.

Не рекомендуется строить зависимости только на основании фиксированных задержек времени. Предпочтительно проверять фактическую готовность данных или статус соответствующего процесса.

5. Определить расписание и параметры выполнения: начальную дату, периодичность, обработку пропущенных интервалов, ограничения параллелизма, тайм-ауты и количество повторных попыток. Выбор настроек необходимо обосновать особенностями источника и допустимым временем обновления данных.

Для длительных или ресурсоемких задач следует ограничить число одновременных запусков и при необходимости использовать очереди или пулы Airflow.

6. Реализовать передачу параметров расчетного интервала между задачами. DAG должен поддерживать ручной запуск для выбранной даты или диапазона и обеспечивать возможность повторной обработки исторических данных без изменения исходного кода.
7. Добавить обработку ошибок и контроль состояния. Необходимо настроить журналирование, уведомление о критическом сбое или фиксацию инцидента, а также продемонстрировать корректное возобновление работы после устранения причины ошибки.
8. Проверить устойчивость оркестрации: выполнить успешный запуск, искусственно вызвать ошибку одной из задач, повторить выполнение только неуспешного участка и провести обратную загрузку данных за ранее выбранный интервал.

Для защиты необходимо предоставить отчет со схемой оркестрации, описанием DAG, расписаний, зависимостей и механизмов повторного выполнения. Также необходимо предоставить исходный код DAG, вспомогательные модули и docker-compose файл.

Обязательным условием является демонстрация веб-интерфейса Apache Airflow: графа зависимостей, истории запусков, журналов задач, ручного запуска с параметрами, обработки ошибочного состояния и успешного повторного выполнения процесса.